

Knowledge Bases

Lecture 10: Modal Logics

dr Anna Maria Radzikowska

Wydział Matematyki i Nauk Informacyjnych

Pokój 504, gmach MiNI

E-mail: A.Radzikowska@mini.pw.edu.pl

What are modalities?

Consider the following sentences:

- (1) *It is raining in London now.*
- (2) **Possibly** *it is raining in London now.*
- (3) **Certainly** *it is raining in London now.*

The sentence (1) expresses some **fact**, while (2) and (3) express a **way** (or **mode**) we say about facts.

(1) has an objective flavour; (2) and (3) have a subjective flavour.

Modalities

Modalities provide extensions to the concept “ α is true” and allow us for modelling notions of necessity, possibility, knowledge, change, time etc.

Examples :

- α is **certainly** true.
- α is **always** true.
- Agent a **believes** that α is true.
- Agent a **knows** that α is true.
- α is true **after all executions** of the program Π .

Modal logics

Modal logics:

- are formal systems for representing and reasoning about modal statements

Modal logics

Modal logics:

- are formal systems for representing and reasoning about modal statements
- have a long and distinguished history (from Aristotle)

Modal logics

Modal logics:

- are formal systems for representing and reasoning about modal statements
- have a long and distinguished history (from Aristotle)
- are interdisciplinary in applications: linguistic, philosophy, computer science.

Modal logics

Modal logics:

- are formal systems for representing and reasoning about modal statements
- have a long and distinguished history (from Aristotle)
- are interdisciplinary in applications: linguistic, philosophy, computer science.

Applications in CS:

- Representing agents' beliefs/knowledge
- Representing events in time
- Representing behaviour of programs.

Modal logics (ML) – Syntax

An *alphabet* of ML consists of:

- a denumerable set Var of *propositional variables*.
- truth constants $\top$ and $\perp$
- logical connectives:
 - binary: $\wedge, \vee, \rightarrow, \equiv$
 - unary: $\neg, \Box, \Diamond$
- brackets (and).

Modal logics (ML) – Syntax

An *alphabet* of ML consists of:

- a denumerable set Var of *propositional variables*.
- truth constants $\top$ and $\perp$
- logical connectives:
 - binary: $\wedge, \vee, \rightarrow, \equiv$
 - unary: $\neg, \Box, \Diamond$
- brackets (and).

Modal formula: $(p \in Var)$

$$\alpha ::= p \mid \neg\alpha \mid \alpha \wedge \beta \mid \alpha \vee \beta \mid \alpha \rightarrow \beta \mid \alpha \equiv \beta \mid \Box\alpha \mid \Diamond\alpha.$$

Syntax of ML (cont.)

Reading of $\Box\alpha$ and $\Diamond\alpha$:

$\Box\alpha$: *certainly* α

$\Diamond\alpha$: *possibly* α .

Syntax of ML (cont.)

Reading of $\Box\alpha$ and $\Diamond\alpha$:

$\Box\alpha$: *certainly* α

$\Diamond\alpha$: *possibly* α .

Precedence of logical connectives (from the highest to the lowest)

$(\neg, \Box, \Diamond)$ $(\wedge, \vee)$ $(\rightarrow, \equiv)$.

Syntax of ML (cont.)

Reading of $\Box\alpha$ and $\Diamond\alpha$:

$\Box\alpha$: *certainly* α

$\Diamond\alpha$: *possibly* α .

Precedence of logical connectives (from the highest to the lowest)

$(\neg, \Box, \Diamond)$ $(\wedge, \vee)$ $(\rightarrow, \equiv)$.

$\Box$ and $\Diamond$ are *dual*: $\Box\alpha \stackrel{def}{=} \neg\Diamond\neg\alpha$.

Readings of modal expressions

$\Box\alpha$	$\Diamond\alpha$
α is certainly true	α is possibly true
α is always true	α is at least once true
Agent a believes in α	α is consistent with a 's beliefs
Agent a knows α	Agent a finds α possible
α holds after all execution of program Π	α holds after at least one possible execution of Π

Semantics of ML

The most popular semantics for ML is a *possible worlds semantics* proposed by S. A. Kripke (1959).

Semantics of ML

The most popular semantics for ML is a *possible worlds semantics* proposed by S. A. Kripke (1959).

A *Kripke structure* is a system

$$\mathcal{K} = (W, R, m)$$

such that

- W is non-empty set of *possible worlds* (states, situations)
- $R \subseteq W \times W$ is the *accessibility relation*;
 $(w, w') \in R$ iff w' is accessible (imaginable, visible) from w
- $m : Var \times W \rightarrow \{0, 1\}$ is the *meaning function*.

Kripke semantics – Valuations

Let $\mathcal{L}$ be a language of ML and let $\mathcal{K} = (W, R, m)$ be a Kripke structure. A **valuation** of a formula $\alpha \in \mathcal{L}$ in $\mathcal{K}$ wrt $w \in W$, in symbols $val_{\mathcal{K}}^w(\alpha)$, is defined as:

$$val_{\mathcal{K}}^w(p) = 1 \quad \text{iff} \quad m(p, w) = 1$$

$$val_{\mathcal{K}}^w(\neg\alpha) = 1 \quad \text{iff} \quad val_{\mathcal{K}}^w(\alpha) = 0$$

$$val_{\mathcal{K}}^w(\alpha \wedge \beta) = 1 \quad \text{iff} \quad \min(val_{\mathcal{K}}^w(\alpha), val_{\mathcal{K}}^w(\beta)) = 1$$

$$val_{\mathcal{K}}^w(\alpha \vee \beta) = 1 \quad \text{iff} \quad \max(val_{\mathcal{K}}^w(\alpha), val_{\mathcal{K}}^w(\beta)) = 1$$

$$val_{\mathcal{K}}^w(\alpha \rightarrow \beta) = 1 \quad \text{iff} \quad val_{\mathcal{K}}^w(\alpha) = 0 \text{ or } val_{\mathcal{K}}^w(\beta) = 1$$

$$val_{\mathcal{K}}^w(\alpha \equiv \beta) = 1 \quad \text{iff} \quad val_{\mathcal{K}}^w(\alpha) = val_{\mathcal{K}}^w(\beta)$$

$$val_{\mathcal{K}}^w(\Box\alpha) = 1 \quad \text{iff} \quad \text{for every } w' \in W, wRw' \implies val_{\mathcal{K}}^{w'}(\alpha) = 1$$

$$val_{\mathcal{K}}^w(\Diamond\alpha) = 1 \quad \text{iff} \quad \text{for some } w' \in W, wRw' \text{ and } val_{\mathcal{K}}^{w'}(\alpha) = 1.$$

Kripke semantics (cont.)

A Kripke structure can be viewed as a **graph**, where W is the set of **nodes**, R is the set of **edges**, and m is information written in nodes.

Kripke semantics (cont.)

A Kripke structure can be viewed as a **graph**, where W is the set of **nodes**, R is the set of **edges**, and m is information written in nodes.

Let $\mathcal{K} = (W, R, m)$ be a Kripke structure and let α be a formula of ML.

Kripke semantics (cont.)

A Kripke structure can be viewed as a **graph**, where W is the set of **nodes**, R is the set of **edges**, and m is information written in nodes.

Let $\mathcal{K} = (W, R, m)$ be a Kripke structure and let α be a formula of ML.

If $val_{\mathcal{K}}^w(\alpha) = 1$ then we write $\boxed{\mathcal{K}, w \models \alpha}$

Kripke semantics (cont.)

A Kripke structure can be viewed as a **graph**, where W is the set of **nodes**, R is the set of **edges**, and m is information written in nodes.

Let $\mathcal{K} = (W, R, m)$ be a Kripke structure and let α be a formula of ML.

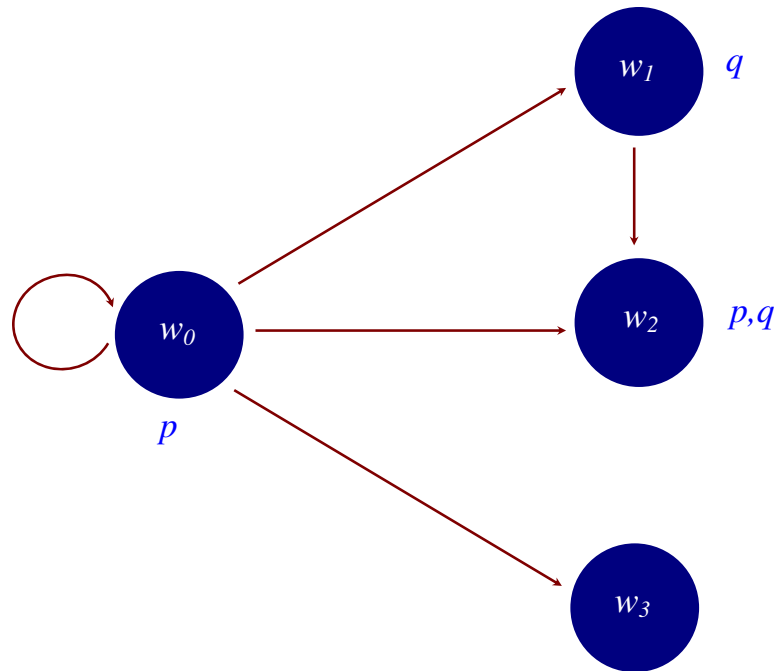
If $val_{\mathcal{K}}^w(\alpha) = 1$ then we write $\boxed{\mathcal{K}, w \models \alpha}$

We say that a formula α is

- **satisfied** in $\mathcal{K}$ wrt $w \in W$ iff $\mathcal{K}, w \models \alpha$.
- **satisfied** in $\mathcal{K}$ iff $\mathcal{K}, w \models \alpha$ for every $w \in W$.
In such case we say that $\mathcal{K}$ is a **model of α** .
- **tautology (valid)** iff every $\mathcal{K}$ is a model of α .

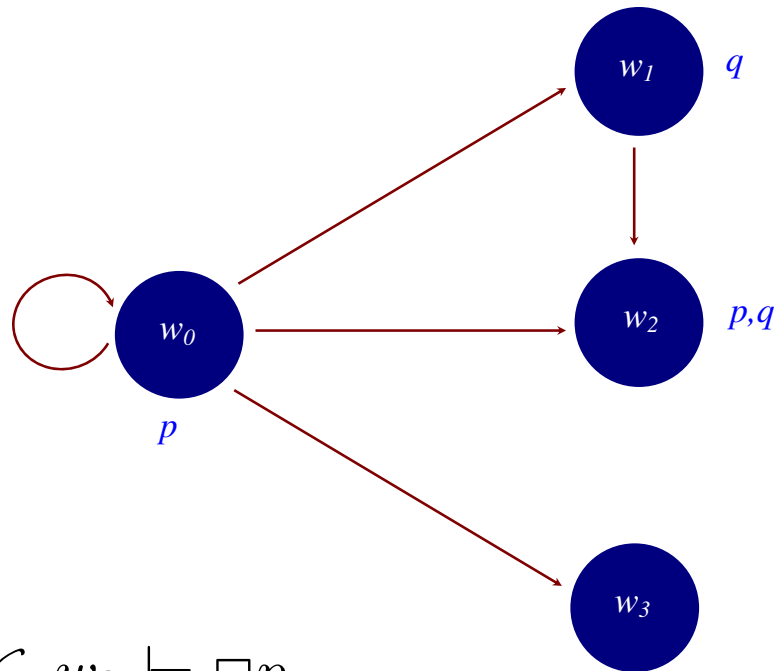
Example 10.1

Let a Kripke structure $\mathcal{K}$ be depicted as:



Example 10.1

Let a Kripke structure $\mathcal{K}$ be depicted as:



$$\mathcal{K}, w_0 \models \Diamond p$$

$$\mathcal{K}, w_1 \models \Box q$$

$$\mathcal{K}, w_2 \models \Box p$$

$$\mathcal{K}, w_2 \models \Box \neg p.$$

Valid and non-valid formulas

Valid formulas:

- $\models \Box(\alpha \rightarrow \beta) \rightarrow (\Box\alpha \rightarrow \Box\beta)$
- $\models (\Box\alpha \wedge \Box(\alpha \rightarrow \beta)) \rightarrow \Box\beta$
- $\models (\Box\alpha \vee \Box\beta) \rightarrow \Box(\alpha \vee \beta)$
- $\models (\Box\alpha \wedge \Box\beta) \equiv \Box(\alpha \wedge \beta)$
- $\models \Diamond(\alpha \vee \beta) \equiv (\Diamond\alpha \vee \Diamond\beta)$
- $\models \Diamond(\alpha \wedge \beta) \rightarrow (\Diamond\alpha \wedge \Diamond\beta).$

Non-valid formulas:

- $\not\models \Box(\alpha \vee \beta) \rightarrow (\Box\alpha \vee \Box\beta)$
- $\not\models (\Diamond\alpha \wedge \Diamond\beta) \rightarrow \Diamond(\alpha \wedge \beta).$

Epistemic logics

Epistemic logics (EL)

- Epistemic logics constitute a family of modal logics used for representing statements of the form “*Agent a knows the fact α* ”.

Epistemic logics (EL)

- Epistemic logics constitute a family of modal logics used for representing statements of the form “*Agent a knows the fact α* ”.
- In EL the box operator $\Box$ is written K (*knowledge operator*) and the above statement is represented as $K_a\alpha$

Epistemic logics (EL)

- Epistemic logics constitute a family of modal logics used for representing statements of the form “*Agent a knows the fact α* ”.
- In EL the box operator $\Box$ is written K (*knowledge operator*) and the above statement is represented as $K_a\alpha$
- According to the semantics of the box operator we have the following interpretation of $K_a\alpha$:

Epistemic logics (EL)

- Epistemic logics constitute a family of modal logics used for representing statements of the form “*Agent a knows the fact α* ”.
- In EL the box operator $\Box$ is written K (*knowledge operator*) and the above statement is represented as $K_a\alpha$
- According to the semantics of the box operator we have the following interpretation of $K_a\alpha$:
An agent a knows the fact α iff in every situation he/she can imagine α holds.

Syntax of EL

- The alphabet of EL is that of the one of ML extended by a denumerable set Ag of **agents** and the set $K = \{K_a : a \in Ag\}$ of knowledge operators.

Syntax of EL

- The alphabet of EL is that of the one of ML extended by a denumerable set Ag of **agents** and the set $K = \{K_a : a \in Ag\}$ of knowledge operators.
- The set of all formulas of EL (**language of EL**) $\mathcal{L}$ is the smallest set of expressions such that
 - $Var \subset \mathcal{L}$
 - if $\alpha, \beta \in \mathcal{L}$ and $a \in Ag$, then $\neg\alpha, \alpha \wedge \beta, \alpha \vee \beta, \alpha \rightarrow \beta, \alpha \equiv \beta, K_a\alpha \in \mathcal{L}$.

Syntax of EL

- The alphabet of EL is that of the one of ML extended by a denumerable set Ag of **agents** and the set $K = \{K_a : a \in Ag\}$ of knowledge operators.
- The set of all formulas of EL (**language of EL**) $\mathcal{L}$ is the smallest set of expressions such that
 - $Var \subset \mathcal{L}$
 - if $\alpha, \beta \in \mathcal{L}$ and $a \in Ag$, then $\neg\alpha, \alpha \wedge \beta, \alpha \vee \beta, \alpha \rightarrow \beta, \alpha \equiv \beta, K_a\alpha \in \mathcal{L}$.

Syntax of EL

- The alphabet of EL is that of the one of ML extended by a denumerable set Ag of **agents** and the set $K = \{K_a : a \in Ag\}$ of knowledge operators.
- The set of all formulas of EL (**language of EL**) $\mathcal{L}$ is the smallest set of expressions such that
 - $Var \subset \mathcal{L}$
 - if $\alpha, \beta \in \mathcal{L}$ and $a \in Ag$, then $\neg\alpha, \alpha \wedge \beta, \alpha \vee \beta, \alpha \rightarrow \beta, \alpha \equiv \beta, K_a\alpha \in \mathcal{L}$.

Epistemic logics (cont.)

- For example,
 - $K_a K_b \alpha$: “The agent a knows that the agent b knows α ”.
 - $\neg K_a K_b K_a \alpha$: “The agent a does not know that the agent b knows that he (i.e. a) knows α ”.

Epistemic logics (cont.)

- For example,
 - $K_a K_b \alpha$: “The agent a knows that the agent b knows α ”.
 - $\neg K_a K_b K_a \alpha$: “The agent a does not know that the agent b knows that he (i.e. a) knows α ”.
- The dual operator to K_a (corresponding to the diamond operator) is the **possibility operator** P_a : $P_a \alpha = \neg K_a \neg \alpha$.
 $P_a \alpha$ is read as “**Agent a finds the fact α possible**”.

Epistemic logics (cont.)

- For example,
 - $K_a K_b \alpha$: “The agent a knows that the agent b knows α ”.
 - $\neg K_a K_b K_a \alpha$: “The agent a does not know that the agent b knows that he (i.e. a) knows α ”.
- The dual operator to K_a (corresponding to the diamond operator) is the **possibility operator** P_a : $P_a \alpha = \neg K_a \neg \alpha$.
 $P_a \alpha$ is read as “**Agent a finds the fact α possible**”.

Epistemic logics (cont.)

- For example,
 - $K_a K_b \alpha$: “The agent a knows that the agent b knows α ”.
 - $\neg K_a K_b K_a \alpha$: “The agent a does not know that the agent b knows that he (i.e. a) knows α ”.
- The dual operator to K_a (corresponding to the diamond operator) is the **possibility operator** P_a : $P_a \alpha = \neg K_a \neg \alpha$.
 $P_a \alpha$ is read as “**Agent a finds the fact α possible**”.

Semantics of EL

A *Kripke structure* for a language $\mathcal{L}$ of EL is a system

$$\mathcal{K} = (W, \{K_a : a \in Ag\}, m)$$

where

- W is a non-empty set of *possible worlds*
- for every agent $a \in Ag$, $K_a \subseteq W \times W$ is a *knowledge relation*
- $m : Var \times W \rightarrow \{0, 1\}$ is the *meaning function*.

Semantics of EL

A *Kripke structure* for a language $\mathcal{L}$ of EL is a system

$$\mathcal{K} = (W, \{K_a : a \in Ag\}, m)$$

where

- W is a non-empty set of *possible worlds*
- for every agent $a \in Ag$, $K_a \subseteq W \times W$ is a *knowledge relation*
- $m : Var \times W \rightarrow \{0, 1\}$ is the *meaning function*.

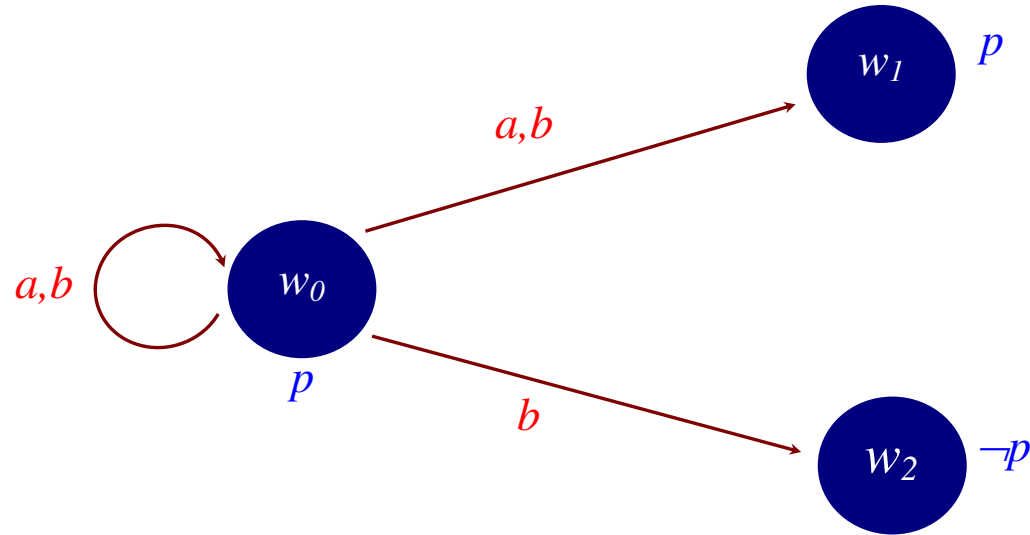
A Kripke structure can be viewed as a **graph**, where W is the set of **myred nodes**, $\{K_a : a \in Ag\}$ is the set of **edges** (labeled by agents), and m is information written in nodes.

Semantics of EL (cont.)

The knowledge operators K_a and the possibility operator P_a are interpreted as $\Box$ and $\Diamond$, respectively. Namely,

- $\mathcal{K}, w \models K_a \alpha$ iff $(\forall w' \in W) w R w' \implies \mathcal{K}, w' \models \alpha$
- $\mathcal{K}, w \models P_a \alpha$ iff $(\exists w' \in W) w R w' \ \& \ \mathcal{K}, w' \models \alpha$.

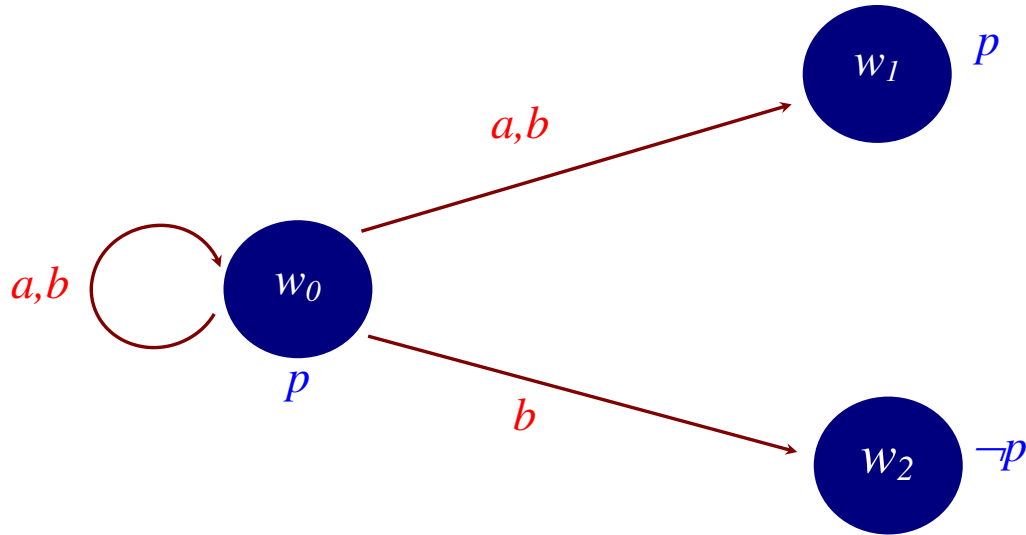
Example 10.2



In w_0 agent b “imagines” all situations (w_0, w_1, w_2) , whereas a only w_0 and w_1 .

Note that $\mathcal{K}, w_0 \models K_a p \wedge \neg K_b p$.

Example 10.2



In w_0 agent b “imagines” all situations (w_0, w_1, w_2) , whereas a only w_0 and w_1 .

Note that $\mathcal{K}, w_0 \models K_a p \wedge \neg K_b p$.

The more situations one can imagine, the more limited his/her knowledge is!

Example 10.3: Playing cards

Consider the following statements:

There is a deck consisting of 3 cards labeled by 1, 2, and 3. Agent a and b each get exactly one of these cards, the third one is left face down.

Example 10.3: Playing cards

Consider the following statements:

There is a deck consisting of 3 cards labeled by 1, 2, and 3. Agent a and b each get exactly one of these cards, the third one is left face down.

We are interested in reasoning about the cards held by agents a and b .

Example 10.3: Playing cards

Consider the following statements:

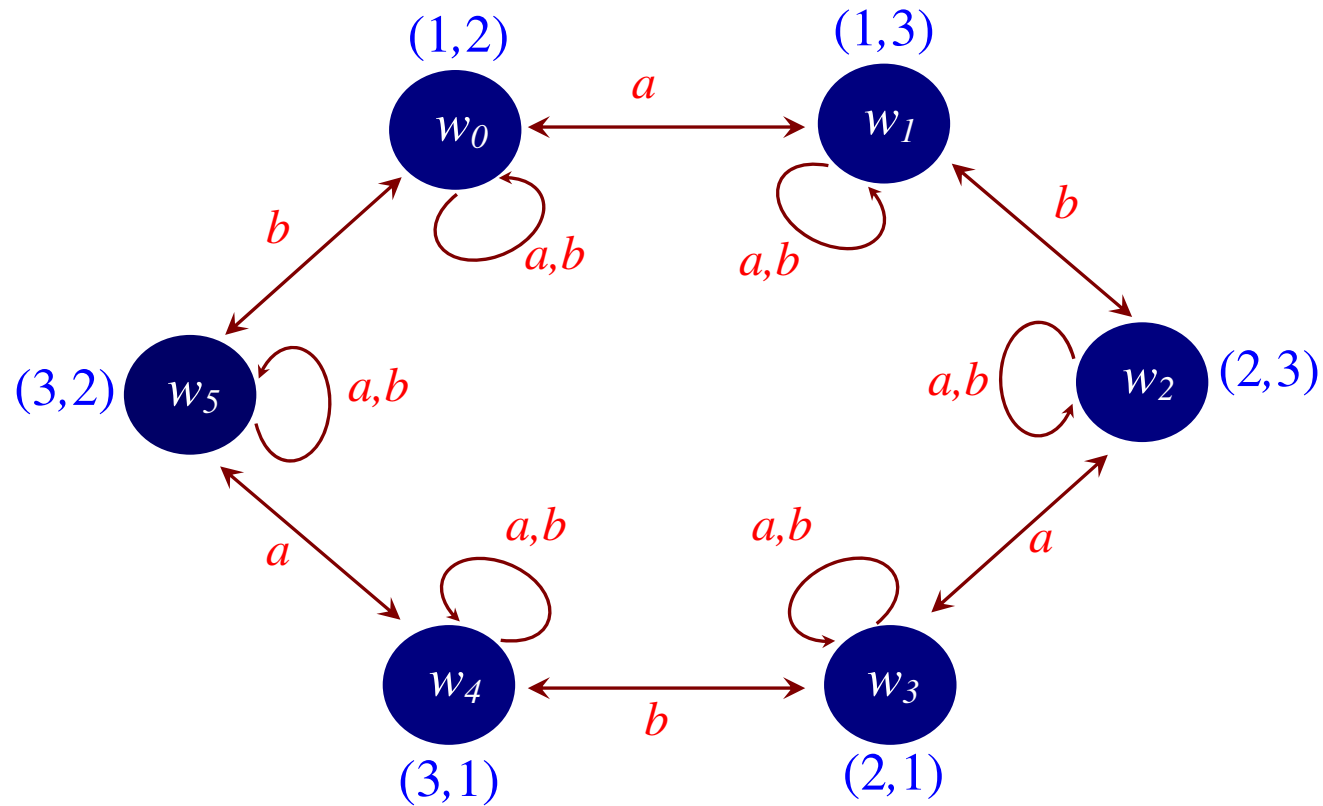
There is a deck consisting of 3 cards labeled by 1, 2, and 3. Agent a and b each get exactly one of these cards, the third one is left face down.

We are interested in reasoning about the cards held by agents a and b .

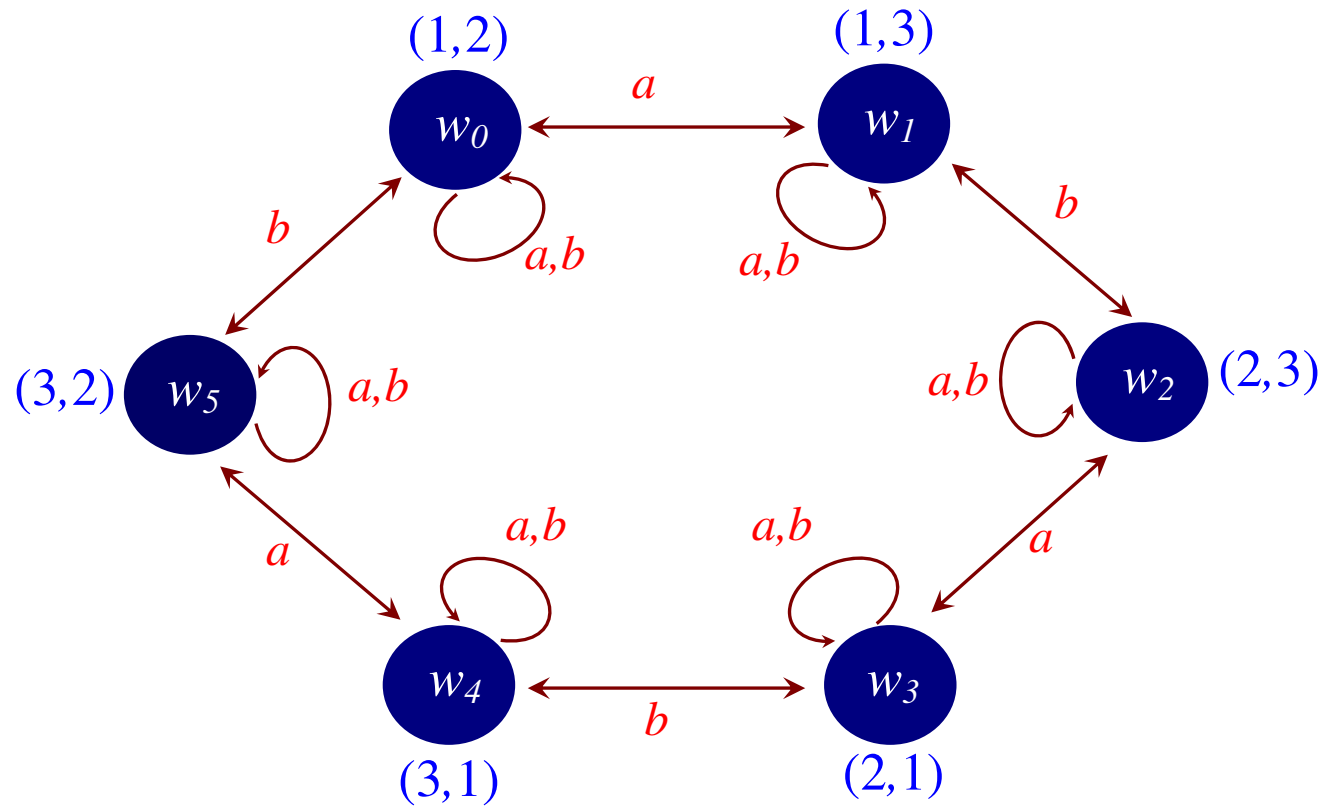
Possible states are described by pairs (i, j) , where i stands for the label of the card held by a and j — by b .

Propositions represent statements “agent x holds card i ”. So we have 6 propositions: $a1$, $a2$, $a3$, $b1$, $b2$, and $b3$.

Example 10.3 (cont.)



Example 10.3 (cont.)



$$\mathcal{K}, w_0 \models (a1 \wedge b2) \wedge K_a(b2 \vee b3) \wedge K_a \neg K_b(a1).$$

Muddy Children Puzzle

Imagine $n \geq 2$ children playing together. During the play $k \geq 1$ children get mud on their foreheads. Each can see the mud on others, but not on his own forehead. Along comes the father and says:

“A least one of you has mud on your forehead”.

Then he asks the question over and over:

“Does any of you know whether he has mud on his forehead?”

Assume that all children are perceptive, intelligent, truthful, and they answer simultaneously.

Muddy Children Puzzle

Imagine $n \geq 2$ children playing together. During the play $k \geq 1$ children get mud on their foreheads. Each can see the mud on others, but not on his own forehead. Along comes the father and says:

“A least one of you has mud on your forehead”.

Then he asks the question over and over:

“Does any of you know whether he has mud on his forehead?”

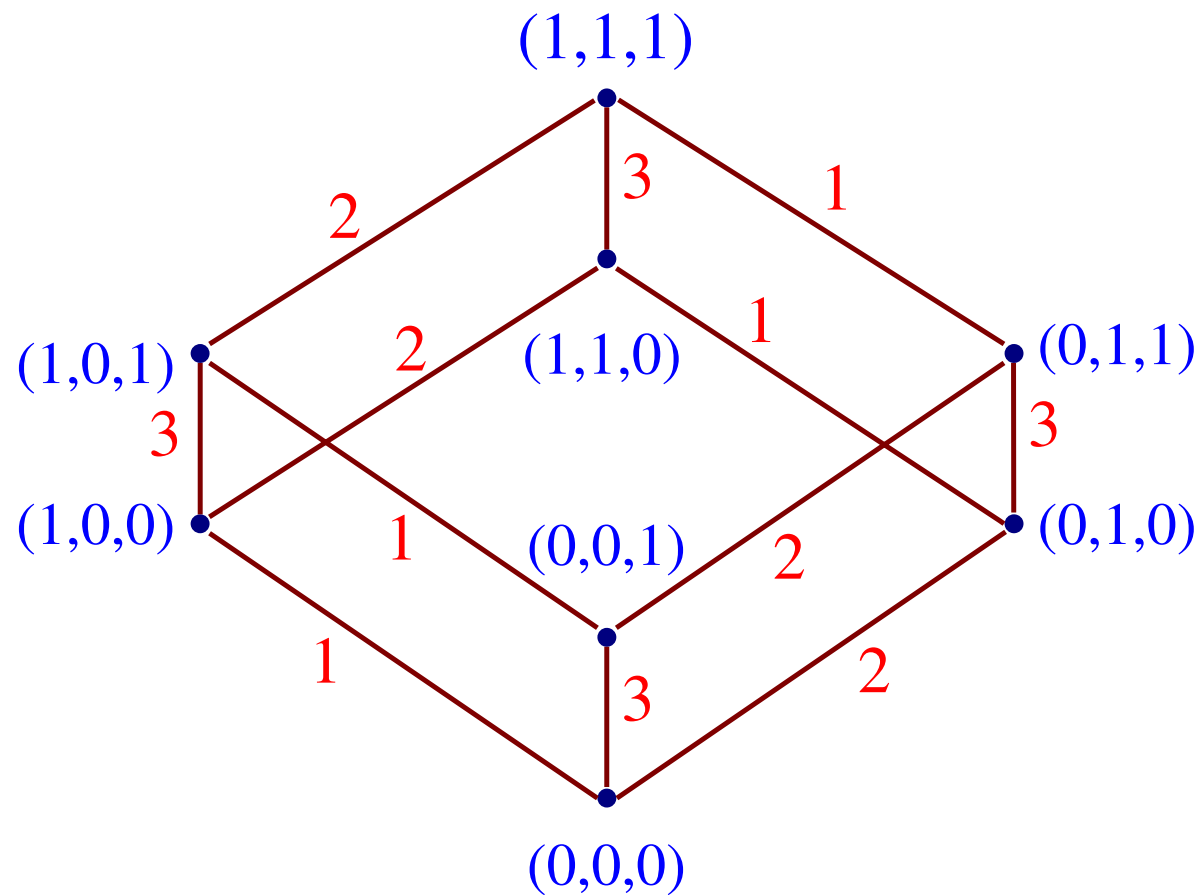
Assume that all children are perceptive, intelligent, truthful, and they answer simultaneously.

How many times does the father ask his question, so that finally all children with dirty foreheads answer YES, I KNOW?

Muddy Children (cont.)

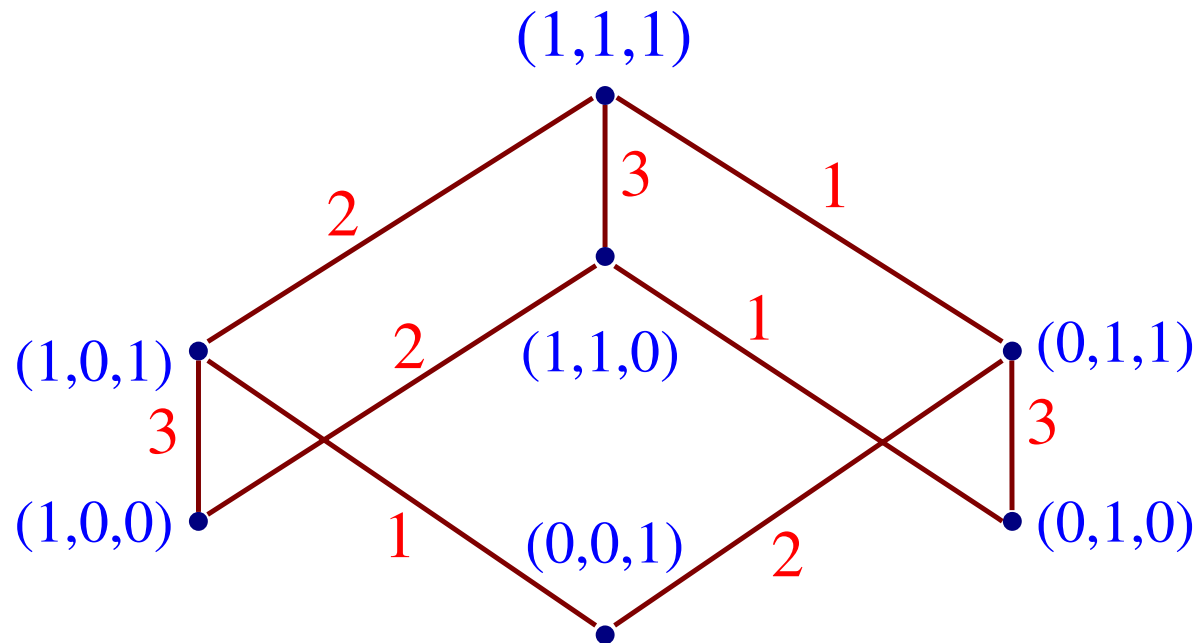
- Consider the case for $n = 3$.
- A *possible state* is described by (c_1, c_2, c_3) , where $c_i \in \{0, 1\}$ represents the statement “*the child i -th has muddy forehead*’.

Muddy Children (cont.)



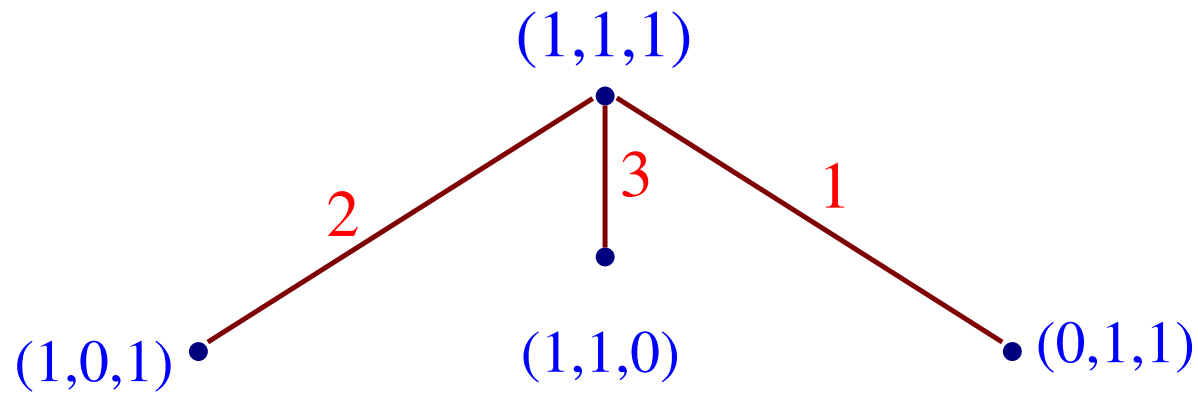
Muddy Children (cont.)

After the father said “*At least one of you has mud on his forehead*”



Muddy Children (cont.)

After the 1st round.



Muddy Children (cont.)

After the 2nd round.

(1,1,1)
•

Types of knowledge

The syntax of EL is typically extended by three additional modal operators representing knowledge of groups of agents. Namely, for $G \subseteq Ag$ and for any formula α (representing facts),

- $E_G\alpha$ — “everybody in the group G knows α ”

Types of knowledge

The syntax of EL is typically extended by three additional modal operators representing knowledge of groups of agents. Namely, for $G \subseteq Ag$ and for any formula α (representing facts),

- $E_G\alpha$ — “everybody in the group G knows α ”
- $D_G\alpha$ — “ α is the distributed knowledge among agents from G ”

Types of knowledge

The syntax of EL is typically extended by three additional modal operators representing knowledge of groups of agents. Namely, for $G \subseteq Ag$ and for any formula α (representing facts),

- $E_G\alpha$ — “everybody in the group G knows α ”
- $D_G\alpha$ — “ α is the distributed knowledge among agents from G ”
- $C_G\alpha$ — “ α is the common knowledge among agents from G ”.

Group knowledge

Let $G \subseteq Ag$ be a group of agents. We say that α is a **group knowledge**, written $E_G\alpha$, iff every agent in G knows α .

Interpretation:

For every Kripke structure $\mathcal{K} = (W, \{K_a, a \in Ag\}, m)$ and for every $w \in W$,

$$\mathcal{K}, w \models E_G\alpha \quad \text{iff} \quad \mathcal{K}, w \models K_a\alpha \text{ for every } a \in G.$$

Distributed knowledge

We say that α is a *distributed knowledge in G* , written $D_G\alpha$, iff it is implied by “joint” knowledge of G .

Distributed knowledge

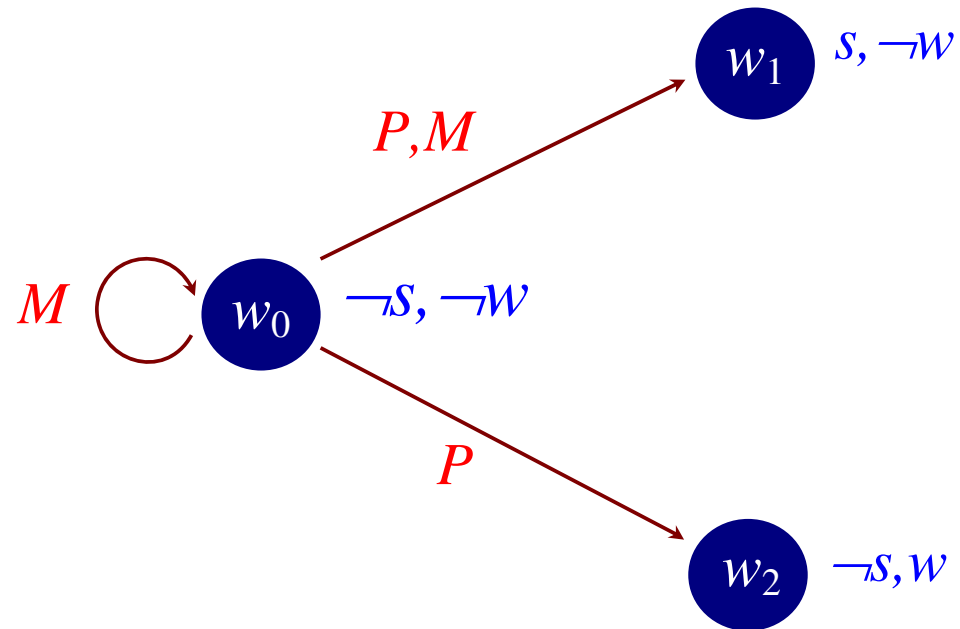
We say that α is a *distributed knowledge in G* , written $D_G\alpha$, iff it is implied by “joint” knowledge of G .

Assume that *Paul* and *Martin* respectively say:

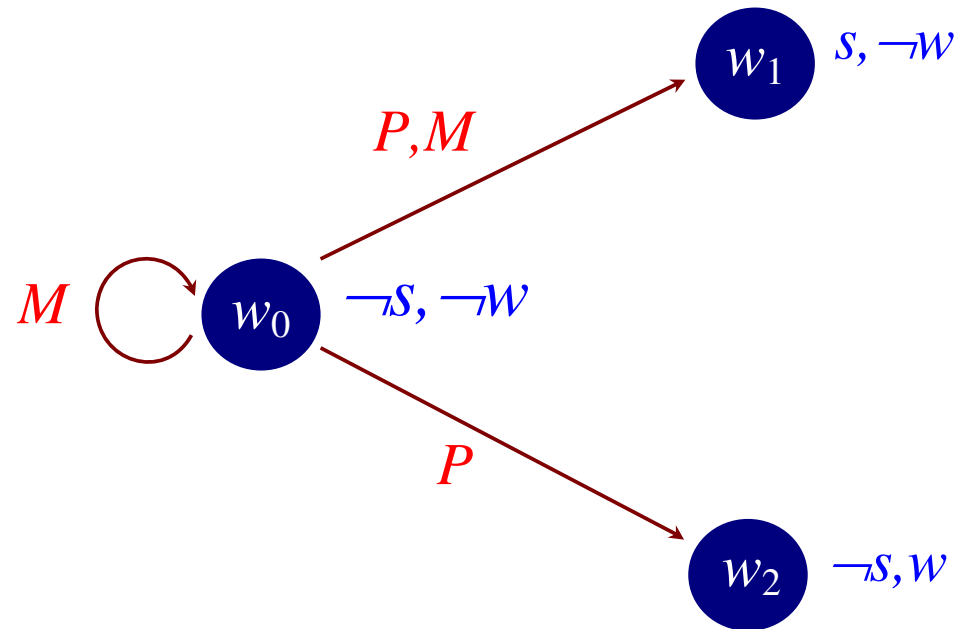
- *Paul*: “*Lennart is in Warsaw or Paris*”.
- *Martin*: “*Lennart is not in Warsaw*”.

Although neither of them knows where *Lennart* is, their joint knowledge implies that “*Lennart is in Paris*”.

Distributed knowledge (cont.)



Distributed knowledge (cont.)



$\mathcal{K}, w_0 \models D_G \alpha$ iff $\mathcal{K}, w' \models \alpha$ for every $w' \in W$ such that $(w, w') \in \bigcap_{a \in G} K_a$.

Common knowledge

We say that α is a *common knowledge* among the group G , written $C_G\alpha$, iff

- every agent from G knows α ,
- every agents from G knows that every agent from G knows α
- etc.

Common knowledge (cont.)

$C_G \alpha$ has the following graph-oriented interpretation.

- A state $w' \in W$ is ***G-reachable from $w \in W$ in $\mathcal{K}$ in $k \geq 1$ steps*** iff there exists a sequence $(w_0, w_1, \dots, w_k)$ of states (from W) such that
 - $w_0 = w$ and $w_k = w'$
 - for every $j = 1, \dots, k - 1$, there is an agent $a \in G$ such that $(w_j, w_{j+1}) \in K_a$.

Common knowledge (cont.)

$C_G\alpha$ has the following graph-oriented interpretation.

- A state $w' \in W$ is **G -reachable from $w \in W$ in $\mathcal{K}$ in $k \geq 1$ steps** iff there exists a sequence $(w_0, w_1, \dots, w_k)$ of states (from W) such that
 - $w_0 = w$ and $w_k = w'$
 - for every $j = 1, \dots, k - 1$, there is an agent $a \in G$ such that $(w_j, w_{j+1}) \in K_a$.
- A state $w' \in W$ is **G -reachable from w in $\mathcal{K}$** iff it is G -reachable from w in $\mathcal{K}$ in k steps for some $k \geq 1$.

Common knowledge (cont.)

$C_G\alpha$ has the following graph-oriented interpretation.

- A state $w' \in W$ is **G -reachable from $w \in W$ in $\mathcal{K}$ in $k \geq 1$ steps** iff there exists a sequence $(w_0, w_1, \dots, w_k)$ of states (from W) such that
 - $w_0 = w$ and $w_k = w'$
 - for every $j = 1, \dots, k - 1$, there is an agent $a \in G$ such that $(w_j, w_{j+1}) \in K_a$.
- A state $w' \in W$ is **G -reachable from w in $\mathcal{K}$** iff it is G -reachable from w in $\mathcal{K}$ in k steps for some $k \geq 1$.

Common knowledge (cont.)

$C_G\alpha$ has the following graph-oriented interpretation.

- A state $w' \in W$ is **G -reachable from $w \in W$ in $\mathcal{K}$ in $k \geq 1$ steps** iff there exists a sequence $(w_0, w_1, \dots, w_k)$ of states (from W) such that
 - $w_0 = w$ and $w_k = w'$
 - for every $j = 1, \dots, k - 1$, there is an agent $a \in G$ such that $(w_j, w_{j+1}) \in K_a$.
- A state $w' \in W$ is **G -reachable from w in $\mathcal{K}$** iff it is G -reachable from w in $\mathcal{K}$ in k steps for some $k \geq 1$.

Common knowledge (cont.)

$C_G\alpha$ has the following graph-oriented interpretation.

- A state $w' \in W$ is **G -reachable from $w \in W$ in $\mathcal{K}$ in $k \geq 1$ steps** iff there exists a sequence $(w_0, w_1, \dots, w_k)$ of states (from W) such that
 - $w_0 = w$ and $w_k = w'$
 - for every $j = 1, \dots, k - 1$, there is an agent $a \in G$ such that $(w_j, w_{j+1}) \in K_a$.
- A state $w' \in W$ is **G -reachable from w in $\mathcal{K}$** iff it is G -reachable from w in $\mathcal{K}$ in k steps for some $k \geq 1$.

$\mathcal{K}, w \models C_G\alpha$ iff $\mathcal{K}, w' \models \alpha$ for every w' G -reachable from w in $\mathcal{K}$.

Common knowledge (cont.)

$C_G\alpha$ has the following graph-oriented interpretation.

- A state $w' \in W$ is **G -reachable from $w \in W$ in $\mathcal{K}$ in $k \geq 1$ steps** iff there exists a sequence $(w_0, w_1, \dots, w_k)$ of states (from W) such that
 - $w_0 = w$ and $w_k = w'$
 - for every $j = 1, \dots, k - 1$, there is an agent $a \in G$ such that $(w_j, w_{j+1}) \in K_a$.
- A state $w' \in W$ is **G -reachable from w in $\mathcal{K}$** iff it is G -reachable from w in $\mathcal{K}$ in k steps for some $k \geq 1$.

$\mathcal{K}, w \models C_G\alpha$ iff $\mathcal{K}, w' \models \alpha$ for every w' G -reachable from w in $\mathcal{K}$.

Properties of knowledge

- **Distribution Axiom:**

$$\models K_a\alpha \wedge K_a(\alpha \rightarrow \beta) \rightarrow K_a\beta$$

Agent knows all logical consequences of his knowledge:

Properties of knowledge

- **Distribution Axiom:**

$$\models K_a\alpha \wedge K_a(\alpha \rightarrow \beta) \rightarrow K_a\beta$$

Agent knows all logical consequences of his knowledge:

- **Knowledge Generalisation Rule:**

$$\frac{\alpha}{K_a\alpha}$$

Agent knows all facts which are true.

Note that this does not mean $\alpha \rightarrow K_a\alpha$ read as “*Whenever α holds then agent knows that*”.

Knowledge Properties (cont.)

- **Knowledge Axiom:**

$$\models K_i \alpha \rightarrow \alpha.$$

Everything that agent knows is true.

Knowledge Properties (cont.)

- **Knowledge Axiom:**

$$\models K_i \alpha \rightarrow \alpha.$$

Everything that agent knows is true.

- **Introspection:**

- **positive:** $\models K_a \alpha \rightarrow K_a(K_a \alpha)$

- **negative:** $\models \neg K_a \alpha \rightarrow K_a(\neg K_a \alpha).$

Knowledge Properties (cont.)

- **Knowledge Axiom:**

$$\models K_i \alpha \rightarrow \alpha.$$

Everything that agent knows is true.

- **Introspection:**

- **positive:** $\models K_a \alpha \rightarrow K_a(K_a \alpha)$

- **negative:** $\models \neg K_a \alpha \rightarrow K_a(\neg K_a \alpha).$

Remark:

- Knowledge Axiom follows from reflexivity of K_a
- Positive Introspection – from transitivity of K_a
- Negative Introspection – from symmetry and transitivity of K_a .

Example 10.4

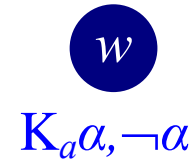
Show that $\models K_a \alpha \rightarrow \alpha$.

Example 10.4

Show that $\models K_a \alpha \rightarrow \alpha$.

Suppose that $\not\models K_a \alpha \rightarrow \alpha$, that is there is $\mathcal{K} = (W, \{K_a : a \in Ag\}, m)$, $w \in W$, and $a \in Ag$ such that

- (1) $\mathcal{K}, w \models K_a \alpha$
- (2) $\mathcal{K}, w \models \neg \alpha$.

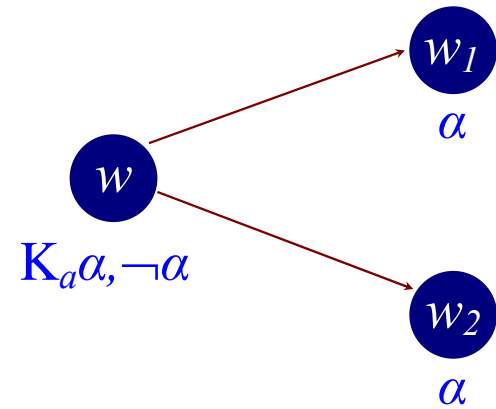


Example 10.4

Show that $\models K_a \alpha \rightarrow \alpha$.

Suppose that $\not\models K_a \alpha \rightarrow \alpha$, that is there is $\mathcal{K} = (W, \{K_a : a \in Ag\}, m)$, $w \in W$, and $a \in Ag$ such that

- (1) $\mathcal{K}, w \models K_a \alpha$
- (2) $\mathcal{K}, w \models \neg \alpha$.



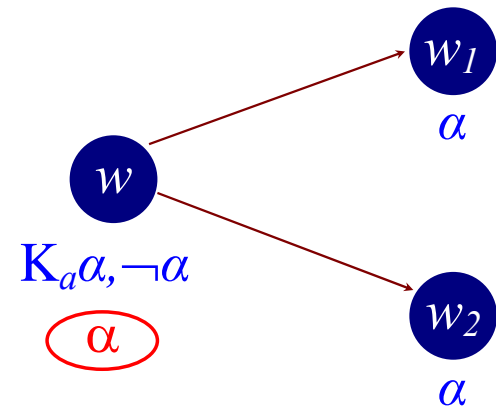
From (1), for every $w_1 \in W$ such that $wK_a w_1$, we have $\mathcal{K}, w_1 \models \alpha$.

Example 10.4

Show that $\models K_a \alpha \rightarrow \alpha$.

Suppose that $\not\models K_a \alpha \rightarrow \alpha$, that is there is $\mathcal{K} = (W, \{K_a : a \in Ag\}, m)$, $w \in W$, and $a \in Ag$ such that

- (1) $\mathcal{K}, w \models K_a \alpha$
- (2) $\mathcal{K}, w \models \neg \alpha$.



From (1), for every $w_1 \in W$ such that $wK_a w_1$, we have $\mathcal{K}, w_1 \models \alpha$.

Since K_a is reflexive, it holds $wK_a w$, so $\mathcal{K}, w \models \alpha$, which contradicts (2).

Propositional Temporal Logic (PDL)

Temporal logic — modal logic of time (Prior, 1957). Modal operators $\Box$ and $\Diamond$ are interpreted here as “*always in the future*” and “*sometime in the future*”, respectively.

Temporal logic — modal logic of time (Prior, 1957). Modal operators $\Box$ and $\Diamond$ are interpreted here as “*always in the future*” and “*sometime in the future*”, respectively.

The alphabet of the language of PDL consists of:

- denumerable set of propositional variables Var
- logical connectives $\neg, \rightarrow$
- modal operators: G (“*going to be*”), F (“*future*”), H (“*has been*”), P (“*past*”), $\bigcirc$ (“*next time*”), and $*$ (“*previous time*”).

Intuitive interpretation

Intuitively, modal formulas are read as:

● $G\alpha$: *always in the future α holds*

Intuitive interpretation

Intuitively, modal formulas are read as:

- $G\alpha$: *always in the future α holds*
- $F\alpha$: *at some time in the future α holds*

Intuitive interpretation

Intuitively, modal formulas are read as:

- $G\alpha$: *always in the future α holds*
- $F\alpha$: *at some time in the future α holds*
- $H\alpha$: *always in the past α was satisfied*

Intuitive interpretation

Intuitively, modal formulas are read as:

- $G\alpha$: *always in the future α holds*
- $F\alpha$: *at some time in the future α holds*
- $H\alpha$: *always in the past α was satisfied*
- $P\alpha$: *at some moment in the past α was satisfied*

Intuitive interpretation

Intuitively, modal formulas are read as:

- $G\alpha$: *always in the future α holds*
- $F\alpha$: *at some time in the future α holds*
- $H\alpha$: *always in the past α was satisfied*
- $P\alpha$: *at some moment in the past α was satisfied*
- $\bigcirc\alpha$: *next moment α will be satisfied*

Intuitive interpretation

Intuitively, modal formulas are read as:

- $G\alpha$: *always in the future α holds*
- $F\alpha$: *at some time in the future α holds*
- $H\alpha$: *always in the past α was satisfied*
- $P\alpha$: *at some moment in the past α was satisfied*
- $\bigcirc\alpha$: *next moment α will be satisfied*
- $*\alpha$: *at previous moment α was satisfied.*

Semantics of PDL

A Kripke structure for the language $\mathcal{L}$ of PDL is a system

$$\Sigma = (T, <, m)$$

where

- T – a denumerable set of *timepoints*
- $<$ – a binary relation on T of *temporal ordering*
- $m : T \times Var \rightarrow \{0, 1\}$ — *valuation* of variables at timepoints.

Semantics of PDL

A Kripke structure for the language $\mathcal{L}$ of PDL is a system

$$\Sigma = (T, <, m)$$

where

- T – a denumerable set of *timepoints*
- $<$ – a binary relation on T of *temporal ordering*
- $m : T \times Var \rightarrow \{0, 1\}$ — *valuation* of variables at timepoints.

Typically it is assumed that $<$ is a *strict partial ordering*, i.e.

- *irreflexive*: $t \not< t$ for every $t \in T$
- *antisymmetric*: $t_1 < t_2 \ \& \ t_2 < t_1 \implies t_1 = t_2$ for all $t_1, t_2 \in T$
- *transitive*: $(t_1 < t_2) \ \& \ (t_2 < t_3) \implies (t_1 < t_3)$ for all $t_1, t_2, t_3 \in T$.

Valuation of formulas

Notation:

$$\begin{aligned} succ(t) &= \{t' \in T : (t < t') \ \& \ (\forall t'' \in T \ t < t'' \implies t' < t'')\} \\ prev(t) &= \{t' \in T : (t' < t) \ \& \ (\forall t'' \in T : t'' < t \implies t'' < t')\}. \end{aligned}$$

Valuation of formulas

Notation:

$$\begin{aligned} succ(t) &= \{t' \in T : (t < t') \ \& \ (\forall t'' \in T \ t < t'' \implies t' < t'')\} \\ prev(t) &= \{t' \in T : (t' < t) \ \& \ (\forall t'' \in T : t'' < t \implies t'' < t')\}. \end{aligned}$$

$\Sigma, t \models \alpha$ — α is valid at timepoint t in the structure Σ . Inductively, it is defined as follows:

• $\Sigma, t \models p$ iff $m(p, t) = 1$

Valuation of formulas

Notation:

$$succ(t) = \{t' \in T : (t < t') \ \& \ (\forall t'' \in T \ t < t'' \implies t' < t'')\}$$

$$prev(t) = \{t' \in T : (t' < t) \ \& \ (\forall t'' \in T : t'' < t \implies t'' < t')\}.$$

$\Sigma, t \models \alpha$ — α is valid at timepoint t in the structure Σ . Inductively, it is defined as follows:

- $\Sigma, t \models p$ iff $m(p, t) = 1$
- $\Sigma, t \models \neg\alpha$ iff $\Sigma, t \not\models \alpha$

Valuation of formulas

Notation:

$$\begin{aligned} \text{succ}(t) &= \{t' \in T : (t < t') \ \& \ (\forall t'' \in T \ t < t'' \implies t' < t'')\} \\ \text{prev}(t) &= \{t' \in T : (t' < t) \ \& \ (\forall t'' \in T : t'' < t \implies t'' < t')\}. \end{aligned}$$

$\Sigma, t \models \alpha$ — α is valid at timepoint t in the structure Σ . Inductively, it is defined as follows:

- $\Sigma, t \models p$ iff $m(p, t) = 1$
- $\Sigma, t \models \neg\alpha$ iff $\Sigma, t \not\models \alpha$
- $\Sigma, t \models \alpha \rightarrow \beta$ iff $\Sigma \not\models \alpha$ or $\Sigma, t \models \beta$

Valuation of formulas

Notation:

$$succ(t) = \{t' \in T : (t < t') \ \& \ (\forall t'' \in T \ t < t'' \implies t' < t'')\}$$

$$prev(t) = \{t' \in T : (t' < t) \ \& \ (\forall t'' \in T : t'' < t \implies t'' < t')\}.$$

$\Sigma, t \models \alpha$ — α is valid at timepoint t in the structure Σ . Inductively, it is defined as follows:

- $\Sigma, t \models p$ iff $m(p, t) = 1$
- $\Sigma, t \models \neg\alpha$ iff $\Sigma, t \not\models \alpha$
- $\Sigma, t \models \alpha \rightarrow \beta$ iff $\Sigma \not\models \alpha$ or $\Sigma, t \models \beta$
- $\Sigma, t \models \bigcirc\alpha$ iff $(\forall t' \in T) \ t' \in succ(t) \implies \Sigma, t' \models \alpha$

Valuation of formulas

Notation:

$$succ(t) = \{t' \in T : (t < t') \ \& \ (\forall t'' \in T \ t < t'' \implies t' < t'')\}$$

$$prev(t) = \{t' \in T : (t' < t) \ \& \ (\forall t'' \in T : t'' < t \implies t'' < t')\}.$$

$\Sigma, t \models \alpha$ — α is valid at timepoint t in the structure Σ . Inductively, it is defined as follows:

- $\Sigma, t \models p$ iff $m(p, t) = 1$
- $\Sigma, t \models \neg\alpha$ iff $\Sigma, t \not\models \alpha$
- $\Sigma, t \models \alpha \rightarrow \beta$ iff $\Sigma \not\models \alpha$ or $\Sigma, t \models \beta$
- $\Sigma, t \models \bigcirc\alpha$ iff $(\forall t' \in T) \ t' \in succ(t) \implies \Sigma, t' \models \alpha$
- $\Sigma, t \models *\alpha$ iff $(\forall t' \in T) \ t' \in prev(t) \implies \Sigma, t' \models \alpha.$

Valuation of formulas (cont.)

- $\Sigma, t \models G\alpha$ iff $(\forall t' \in T) (t < t') \implies \Sigma, t' \models \alpha$

Valuation of formulas (cont.)

- $\Sigma, t \models G\alpha$ iff $(\forall t' \in T) (t < t') \implies \Sigma, t' \models \alpha$
- $\Sigma, t \models F\alpha$ iff $(\exists t' \in T) (t < t') \ \& \ \Sigma, t' \models \alpha$

Valuation of formulas (cont.)

- $\Sigma, t \models G\alpha$ iff $(\forall t' \in T) (t < t') \implies \Sigma, t' \models \alpha$
- $\Sigma, t \models F\alpha$ iff $(\exists t' \in T) (t < t') \ \& \ \Sigma, t' \models \alpha$
- $\Sigma, t \models H\alpha$ iff $(\forall t' \in T) (t' < t) \implies \Sigma, t' \models \alpha$

Valuation of formulas (cont.)

- $\Sigma, t \models G\alpha$ iff $(\forall t' \in T) (t < t') \implies \Sigma, t' \models \alpha$
- $\Sigma, t \models F\alpha$ iff $(\exists t' \in T) (t < t') \ \& \ \Sigma, t' \models \alpha$
- $\Sigma, t \models H\alpha$ iff $(\forall t' \in T) (t' < t) \implies \Sigma, t' \models \alpha$
- $\Sigma, t \models P\alpha$ iff $(\exists t' \in T) (t' < t) \ \& \ \Sigma, t' \models \alpha$

Propositional Dynamic Logic PDL

Syntax of PDL

The alphabet of a language of PDL is that of PL enriched by:

- $\mathcal{At}$ — denumerable set of *atomic actions*
- $?, +, ;, *$ — *action constructors*
- $[], \langle \rangle$ — modal operators.

Syntax of PDL

The alphabet of a language of PDL is that of PL enriched by:

- $\mathcal{At}$ — denumerable set of *atomic actions*
- $?, +, ;, *$ — *action constructors*
- $[], \langle \rangle$ — modal operators.

Basing on $\mathcal{At}$ we build *compound actions* (*actions*, for short) using action constructors. The set of all actions is denoted by $\mathcal{A}$.

Syntax of PDL

The alphabet of a language of PDL is that of PL enriched by:

- $\mathcal{At}$ — denumerable set of **atomic actions**
- $?, +, ;, *$ — **action constructors**
- $[], \langle \rangle$ — modal operators.

Basing on $\mathcal{At}$ we build **compound actions** (**actions**, for short) using action constructors. The set of all actions is denoted by $\mathcal{A}$.

The language $\mathcal{L}$ of PDL is the smallest set of expressions such that

- $Var \subseteq \mathcal{L}$
- $\alpha, \beta \in \mathcal{L} \ \& \ a \in \mathcal{A} \implies \neg\alpha, \alpha \rightarrow \beta, [a]\alpha, \langle a \rangle\alpha \in \mathcal{L}.$

Syntax of PDL

The alphabet of a language of PDL is that of PL enriched by:

- $\mathcal{At}$ — denumerable set of **atomic actions**
- $?, +, ;, *$ — **action constructors**
- $[], \langle \rangle$ — modal operators.

Basing on $\mathcal{At}$ we build **compound actions** (**actions**, for short) using action constructors. The set of all actions is denoted by $\mathcal{A}$.

The language $\mathcal{L}$ of PDL is the smallest set of expressions such that

- $Var \subseteq \mathcal{L}$
- $\alpha, \beta \in \mathcal{L} \ \& \ a \in \mathcal{A} \implies \neg\alpha, \alpha \rightarrow \beta, [a]\alpha, \langle a \rangle\alpha \in \mathcal{L}.$

Intuitively, for every $a \in \mathcal{A}$ and for every $\alpha \in \mathcal{L}$:

- $[a]\alpha$: “if a is performed, then after its execution α always holds”
- $\langle a \rangle\alpha$: “the action a is performed and after its execution α may hold”.

Syntax of PDL (cont.)

The set of actions is the smallest set of expressions such that

- $\mathcal{A}t \subset \mathcal{A}$
- if $a, b \in \mathcal{A}$ and $\alpha \in \mathcal{L}$, then
 - $a; b \in \mathcal{A}$
 - $a + b \in \mathcal{A}$
 - $\alpha? : a, b \in \mathcal{A}$ (conditional action)
 - $a* : \alpha \in \mathcal{A}$ (iteration)

Syntax of PDL (cont.)

The set of actions is the smallest set of expressions such that

- $At \subset A$
- if $a, b \in \mathcal{A}$ and $\alpha \in \mathcal{L}$, then
 - $a; b \in \mathcal{A}$
 - $a + b \in \mathcal{A}$
 - $\alpha? : a, b \in \mathcal{A}$ (conditional action)
 - $a* : \alpha \in \mathcal{A}$ (iteration)

Intuitively:

- $a; b$ — the sequence of actions
- $a + b$ — nondeterministic choice (either a or b is performed)
- $\alpha : a, b$ — if α then a else b
- $a* : \alpha$ — while $\neg\alpha$ do a .

Semantics of PDL

The *Kripke structure* for the language $\mathcal{L}$ of PDL is a system $\Sigma = (S, R, m)$, where

- $S \neq \emptyset$ is a set of *possible states*
- $R : \mathcal{A}t \rightarrow 2^{S \times S}$; for $a \in \mathcal{A}t$, $R(a)$ is a set of pairs (s_1, s_2) of states such that the performance of a in s_1 leads to s_2
- $m : Var \times S \rightarrow \{0, 1\}$ — valuation of variables at states.

Semantics of PDL

The *Kripke structure* for the language $\mathcal{L}$ of PDL is a system $\Sigma = (S, R, m)$, where

- $S \neq \emptyset$ is a set of *possible states*
- $R : \mathcal{At} \rightarrow 2^{S \times S}$; for $a \in \mathcal{At}$, $R(a)$ is a set of pairs (s_1, s_2) of states such that the performance of a in s_1 leads to s_2
- $m : Var \times S \rightarrow \{0, 1\}$ — valuation of variables at states.

Given a structure $\Sigma = (S, R, m)$ and a state $s \in S$, $\Sigma, s \models \alpha$ stands for:
“ α holds in Σ wrt s ”.

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$
- $(s, s') \in R^*(\alpha? : a, b)$ iff
 - $\Sigma, s \models \alpha \implies (s, s') \in R^*(a)$
 - $\Sigma, s \not\models \alpha \implies (s, s') \in R^*(b)$

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$
- $(s, s') \in R^*(\alpha? : a, b)$ iff
 - $\Sigma, s \models \alpha \implies (s, s') \in R^*(a)$
 - $\Sigma, s \not\models \alpha \implies (s, s') \in R^*(b)$
- $(s, s') \in R^*(a* : \alpha)$ iff $(\exists n \in \mathbb{N}) (\exists s_0, s_1, \dots, s_n \in S)$
 - $s_0 = s$ and $s_n = s'$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies (s_i, s_{i+1}) \in R^*(a)$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies \Sigma, s_i \models \neg \alpha$
 - $\Sigma, s' \models \alpha$.

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$
- $(s, s') \in R^*(\alpha? : a, b)$ iff
 - $\Sigma, s \models \alpha \implies (s, s') \in R^*(a)$
 - $\Sigma, s \not\models \alpha \implies (s, s') \in R^*(b)$
- $(s, s') \in R^*(a^* : \alpha)$ iff $(\exists n \in \mathbb{N}) (\exists s_0, s_1, \dots, s_n \in S)$
 - $s_0 = s$ and $s_n = s'$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies (s_i, s_{i+1}) \in R^*(a)$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies \Sigma, s_i \models \neg \alpha$
 - $\Sigma, s' \models \alpha$.

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$
- $(s, s') \in R^*(\alpha? : a, b)$ iff
 - $\Sigma, s \models \alpha \implies (s, s') \in R^*(a)$
 - $\Sigma, s \not\models \alpha \implies (s, s') \in R^*(b)$
- $(s, s') \in R^*(a^* : \alpha)$ iff $(\exists n \in \mathbb{N}) (\exists s_0, s_1, \dots, s_n \in S)$
 - $s_0 = s$ and $s_n = s'$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies (s_i, s_{i+1}) \in R^*(a)$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies \Sigma, s_i \models \neg \alpha$
 - $\Sigma, s' \models \alpha$.

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$
- $(s, s') \in R^*(\alpha? : a, b)$ iff
 - $\Sigma, s \models \alpha \implies (s, s') \in R^*(a)$
 - $\Sigma, s \not\models \alpha \implies (s, s') \in R^*(b)$
- $(s, s') \in R^*(a* : \alpha)$ iff $(\exists n \in \mathbb{N}) (\exists s_0, s_1, \dots, s_n \in S)$
 - $s_0 = s$ and $s_n = s'$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies (s_i, s_{i+1}) \in R^*(a)$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies \Sigma, s_i \models \neg \alpha$
 - $\Sigma, s' \models \alpha$.

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$
- $(s, s') \in R^*(\alpha? : a, b)$ iff
 - $\Sigma, s \models \alpha \implies (s, s') \in R^*(a)$
 - $\Sigma, s \not\models \alpha \implies (s, s') \in R^*(b)$
- $(s, s') \in R^*(a^* : \alpha)$ iff $(\exists n \in \mathbb{N}) (\exists s_0, s_1, \dots, s_n \in S)$
 - $s_0 = s$ and $s_n = s'$
 - $(\forall i \in \mathbb{N}) 0 \leq i < n \implies (s_i, s_{i+1}) \in R^*(a)$
 - $(\forall i \in \mathbb{N}) 0 \leq i < n \implies \Sigma, s_i \models \neg \alpha$
 - $\Sigma, s' \models \alpha$.

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$
- $(s, s') \in R^*(\alpha? : a, b)$ iff
 - $\Sigma, s \models \alpha \implies (s, s') \in R^*(a)$
 - $\Sigma, s \not\models \alpha \implies (s, s') \in R^*(b)$
- $(s, s') \in R^*(a^* : \alpha)$ iff $(\exists n \in \mathbb{N}) (\exists s_0, s_1, \dots, s_n \in S)$
 - $s_0 = s$ and $s_n = s'$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies (s_i, s_{i+1}) \in R^*(a)$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies \Sigma, s_i \models \neg \alpha$
 - $\Sigma, s' \models \alpha$.

Semantics of PDL (cont.)

The mapping $R : \mathcal{At} \rightarrow 2^{S \times S}$ is generalised into the mapping $R^* : \mathcal{A} \rightarrow 2^{S \times S}$ as follows:

- $R^*(a) = R(a)$ for every $a \in \mathcal{At}$
- $(s, s') \in R^*(a; b)$ iff $(\exists s'' \in S) (s, s'') \in R^*(a) \ \& \ (s'', s') \in R^*(b)$
- $R^*(a + b) = R^*(a) \cup R^*(b)$
- $(s, s') \in R^*(\alpha? : a, b)$ iff
 - $\Sigma, s \models \alpha \implies (s, s') \in R^*(a)$
 - $\Sigma, s \not\models \alpha \implies (s, s') \in R^*(b)$
- $(s, s') \in R^*(a^* : \alpha)$ iff $(\exists n \in \mathbb{N}) (\exists s_0, s_1, \dots, s_n \in S)$
 - $s_0 = s$ and $s_n = s'$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies (s_i, s_{i+1}) \in R^*(a)$
 - $(\forall i \in \mathbb{N}) \ 0 \leq i < n \implies \Sigma, s_i \models \neg \alpha$
 - $\Sigma, s' \models \alpha$.

Valuation of formulas in PDL

Let $\Sigma = (S, R, m)$ and $s \in S$.

• $\Sigma, s \models p$ iff $m(s, p) = 1$

Valuation of formulas in PDL

Let $\Sigma = (S, R, m)$ and $s \in S$.

• $\Sigma, s \models p$ iff $m(s, p) = 1$

• $\Sigma, s \models \neg\alpha$ iff $\Sigma, s \not\models \alpha$

Valuation of formulas in PDL

Let $\Sigma = (S, R, m)$ and $s \in S$.

- $\Sigma, s \models p$ iff $m(s, p) = 1$
- $\Sigma, s \models \neg\alpha$ iff $\Sigma, s \not\models \alpha$
- $\Sigma, s \models \alpha \rightarrow \beta$ iff $\Sigma, s \models \neg\alpha$ & $\Sigma, s \models \beta$

Valuation of formulas in PDL

Let $\Sigma = (S, R, m)$ and $s \in S$.

- $\Sigma, s \models p$ iff $m(s, p) = 1$
- $\Sigma, s \models \neg\alpha$ iff $\Sigma, s \not\models \alpha$
- $\Sigma, s \models \alpha \rightarrow \beta$ iff $\Sigma, s \models \neg\alpha$ & $\Sigma, s \models \beta$
- $\Sigma, s \models [a]\alpha$ iff $(\forall s' \in S) (s, s') \in R^*(a) \implies \Sigma, s' \models \alpha$

Valuation of formulas in PDL

Let $\Sigma = (S, R, m)$ and $s \in S$.

- $\Sigma, s \models p$ iff $m(s, p) = 1$
- $\Sigma, s \models \neg\alpha$ iff $\Sigma, s \not\models \alpha$
- $\Sigma, s \models \alpha \rightarrow \beta$ iff $\Sigma, s \models \neg\alpha$ & $\Sigma, s \models \beta$
- $\Sigma, s \models [a]\alpha$ iff $(\forall s' \in S) (s, s') \in R^*(a) \implies \Sigma, s' \models \alpha$
- $\Sigma, s \models \langle a \rangle \alpha$ iff $(\exists s' \in S) (s, s') \in R^*(a) \text{ \& } \Sigma, s' \models \alpha$.

Belief–Desires–Intentions Structures (BDI)

BDI systems

BDI systems are logical systems for modeling agents, their knowledge, beliefs, intentions, and actions.

BDI systems

BDI systems are logical systems for modeling agents, their knowledge, beliefs, intentions, and actions.

Agent's knowledge, beliefs and desires are modeled by means of the suitable binary relations on the set of possible states. Specifically, for every agent a , we have three relations $K, B, D \subseteq S \times S$ viewed as:

- $(s, s') \in K(a)$ iff s' is K-accessible from s for a (wrt his knowledge)
- $(s, s') \in B(a)$ iff s' is B-accessible from s for a (wrt his beliefs)
- $(s, s') \in D(a)$ iff s' is D-accessible from s for a (wrt his desires).

BDI systems (cont.)

Typically, it is assumed that

• the relation B is

- *serial*: $(\forall s \in S) (\exists s' \in S) (s, s') \in B(a)$
- *Euclidean*: $(\forall s, s', s'' \in S)$
 $(s, s') \in B(a) \ \& \ (s, s'') \in B(a) \implies (s', s'') \in B(a)$
- *symmetric*: $(\forall s, s' \in S) (s, s') \in B(a) \implies (s', s) \in B(a).$

BDI systems (cont.)

Typically, it is assumed that

- the relation B is
 - *serial*: $(\forall s \in S) (\exists s' \in S) (s, s') \in B(a)$
 - *Euclidean*: $(\forall s, s', s'' \in S)$
 $(s, s') \in B(a) \ \& \ (s, s'') \in B(a) \implies (s', s'') \in B(a)$
 - *symmetric*: $(\forall s, s' \in S) (s, s') \in B(a) \implies (s', s) \in B(a).$
- the relation D is serial only.

BDI systems (cont.)

Typically, it is assumed that

- the relation B is
 - *serial*: $(\forall s \in S) (\exists s' \in S) (s, s') \in B(a)$
 - *Euclidean*: $(\forall s, s', s'' \in S)$
 $(s, s') \in B(a) \ \& \ (s, s'') \in B(a) \implies (s', s'') \in B(a)$
 - *symmetric*: $(\forall s, s' \in S) (s, s') \in B(a) \implies (s', s) \in B(a)$.
- the relation D is serial only.

For modeling of agent's intentions there are two approaches:

- yet another relation $I(a) \subseteq S \times S$: *α is a 's intention iff it holds in all states I -accessible from s for a*
- *α is a 's intention in $s \in S$ iff it holds at some point on each path started in s .*

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non-empty denumerable set Var of *propositional variables*

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non-empty denumerable set Var of *propositional variables*
- a non-empty denumerable set Ag of *agents*

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non-empty denumerable set Var of *propositional variables*
- a non-empty denumerable set Ag of *agents*
- a non-empty denumerable set At of *atomic actions*

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non-empty denumerable set Var of *propositional variables*
- a non-empty denumerable set Ag of *agents*
- a non-empty denumerable set At of *atomic actions*
- logical connectives $\neg, \rightarrow$

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non-empty denumerable set Var of **propositional variables**
- a non-empty denumerable set Ag of **agents**
- a non-empty denumerable set At of **atomic actions**
- logical connectives $\neg, \rightarrow$
- modal operators B (**beliefs**) and D (**desires**)

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non-empty denumerable set Var of *propositional variables*
- a non-empty denumerable set Ag of *agents*
- a non-empty denumerable set At of *atomic actions*
- logical connectives $\neg, \rightarrow$
- modal operators B (*beliefs*) and D (*desires*)
- temporal operators $\bigcirc, F$ (*in the future*), G (*going to be*)

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non–empty denumerable set Var of *propositional variables*
- a non–empty denumerable set Ag of *agents*
- a non–empty denumerable set At of *atomic actions*
- logical connectives $\neg, \rightarrow$
- modal operators B (*beliefs*) and D (*desires*)
- temporal operators $\bigcirc, F$ (*in the future*), G (*going to be*)
- dynamic operators $[], \langle \rangle$

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non-empty denumerable set Var of *propositional variables*
- a non-empty denumerable set Ag of *agents*
- a non-empty denumerable set At of *atomic actions*
- logical connectives $\neg, \rightarrow$
- modal operators B (*beliefs*) and D (*desires*)
- temporal operators $\bigcirc, F$ (*in the future*), G (*going to be*)
- dynamic operators $[], \langle \rangle$
- path operators I (*intentions*), E .

Syntax of BDI

An alphabet of a language of BDI consists of:

- a non-empty denumerable set Var of **propositional variables**
- a non-empty denumerable set Ag of **agents**
- a non-empty denumerable set At of **atomic actions**
- logical connectives $\neg, \rightarrow$
- modal operators B (**beliefs**) and D (**desires**)
- temporal operators $\bigcirc, F$ (**in the future**), G (**going to be**)
- dynamic operators $[], \langle \rangle$
- path operators I (**intentions**), E .

Intuitively, $E\alpha$ means: “ α holds on some path started in the state s ”.

BDI formulas

There are 2 types of formulas in BDI languages:

- *path formulas* (subset of the language) — interpreted in stated wrt paths started these states
- *formulas* — interpreted in stated only.

BDI formulas

There are 2 types of formulas in BDI languages:

- *path formulas* (subset of the language) — interpreted in stated wrt paths started these states
- *formulas* — interpreted in stated only.

The set $\mathcal{F}_\pi$ of path formulas is the smallest set of expressions such that

- $Var \subset \mathcal{F}_\pi$
- if $\alpha, \beta \in \mathcal{F}_\pi$, $a \in \mathcal{At}$, and $x \in \mathcal{Ag}$, then
$$\neg\alpha, \alpha \rightarrow \beta, \bigcirc\alpha, G\alpha, F\alpha, x[a]\alpha, x\langle a\rangle\alpha \in \mathcal{F}_\pi.$$

BDI formulas

There are 2 types of formulas in BDI languages:

- *path formulas* (subset of the language) — interpreted in stated wrt paths started these states
- *formulas* — interpreted in stated only.

The set $\mathcal{F}_\pi$ of path formulas is the smallest set of expressions such that

- $Var \subset \mathcal{F}_\pi$
- if $\alpha, \beta \in \mathcal{F}_\pi$, $a \in \mathcal{At}$, and $x \in \mathcal{Ag}$, then
$$\neg\alpha, \alpha \rightarrow \beta, \bigcirc\alpha, G\alpha, F\alpha, x[a]\alpha, x\langle a\rangle\alpha \in \mathcal{F}_\pi.$$

The set $\mathcal{L}$ of all formulas is the smallest set of expressions such that

- $\mathcal{F}_\pi \subset \mathcal{L}$
- $\alpha \in \mathcal{L}, x \in \mathcal{Ag} \implies B(x, \alpha), D(x, \alpha), E(x, \alpha), I(x, \alpha) \in \mathcal{L}.$

Semantics of BDI — notations

Notation:

- $S^* = \{(s_1, s_2, \dots,) : s_i \in S \text{ \& } s_i < s_{i+1} \text{ for every } i \in N\}$,
i.e. S^* is the set of all sequences (finite, infinite) of states.

Semantics of BDI — notations

Notation:

- $S^* = \{(s_1, s_2, \dots) : s_i \in S \text{ \& } s_i < s_{i+1} \text{ for every } i \in N\}$,
i.e. S^* is the set of all sequences (finite, infinite) of states.
- S_s^* – the set of paths starting in the state s .

Semantics of BDI — notations

Notation:

- $S^* = \{(s_1, s_2, \dots) : s_i \in S \text{ \& } s_i < s_{i+1} \text{ for every } i \in N\}$,
i.e. S^* is the set of all sequences (finite, infinite) of states.
- S_s^* – the set of paths starting in the state s .
- for $s \in S$ and $\wp \in S^*$, $s' \in succ(\wp, s)$ iff
 $(s' \in \wp) \text{ \& } (s < s') \text{ \& } (\forall s'' \in S) (s'' \in \wp) \text{ \& } (s < s'') \implies (s'' \not< s')$.

Semantics of BDI

A *Kripke structure* for the language $\mathcal{L}$ of BDI is a system

$$\Sigma = (S, <, B, D, I, m, \mu)$$

such that

- $S \neq \emptyset$ is a set of states

Semantics of BDI

A *Kripke structure* for the language $\mathcal{L}$ of BDI is a system

$$\Sigma = (S, <, B, D, I, m, \mu)$$

such that

- $S \neq \emptyset$ is a set of states
- $< \subseteq S \times S$ is a temporal ordering

Semantics of BDI

A *Kripke structure* for the language $\mathcal{L}$ of BDI is a system

$$\Sigma = (S, <, B, D, I, m, \mu)$$

such that

- $S \neq \emptyset$ is a set of states
- $< \subseteq S \times S$ is a temporal ordering
- $B, D : Ag \rightarrow 2^{S \times S}; B \subseteq D$ (*an agent's desires are his beliefs*)

Semantics of BDI

A *Kripke structure* for the language $\mathcal{L}$ of BDI is a system

$$\Sigma = (S, <, B, D, I, m, \mu)$$

such that

- $S \neq \emptyset$ is a set of states
- $< \subseteq S \times S$ is a temporal ordering
- $B, D : Ag \rightarrow 2^{S \times S}$; $B \subseteq D$ (*an agent's desires are his beliefs*)
- $I : Ag \times S \rightarrow 2^{S^*}$ — for $x \in Ag$ and for $s \in S$, $I(x, s)$ is the set of “preferable paths” for x in s ; it is assumed that $I(x, s) \neq \emptyset$ and for every $\wp \in I(x, s)$, $\wp \in S_s^*$

Semantics of BDI

A *Kripke structure* for the language $\mathcal{L}$ of BDI is a system

$$\Sigma = (S, <, B, D, I, m, \mu)$$

such that

- $S \neq \emptyset$ is a set of states
- $< \subseteq S \times S$ is a temporal ordering
- $B, D : Ag \rightarrow 2^{S \times S}$; $B \subseteq D$ (*an agent's desires are his beliefs*)
- $I : Ag \times S \rightarrow 2^{S^*}$ — for $x \in Ag$ and for $s \in S$, $I(x, s)$ is the set of “preferable paths” for x in s ; it is assumed that $I(x, s) \neq \emptyset$ and for every $\wp \in I(x, s)$, $\wp \in S_s^*$
- $m : Var \times S \rightarrow \{0, 1\}$

Semantics of BDI

A *Kripke structure* for the language $\mathcal{L}$ of BDI is a system

$$\Sigma = (S, <, B, D, I, m, \mu)$$

such that

- $S \neq \emptyset$ is a set of states
- $< \subseteq S \times S$ is a temporal ordering
- $B, D : Ag \rightarrow 2^{S \times S}$; $B \subseteq D$ (*an agent's desires are his beliefs*)
- $I : Ag \times S \rightarrow 2^{S^*}$ — for $x \in Ag$ and for $s \in S$, $I(x, s)$ is the set of “preferable paths” for x in s ; it is assumed that $I(x, s) \neq \emptyset$ and for every $\wp \in I(x, s)$, $\wp \in S_s^*$
- $m : Var \times S \rightarrow \{0, 1\}$
- $\mu : Ag \times \mathcal{A}t \rightarrow 2^{S \times S}$; for $x \in Ag$ and $a \in \mathcal{A}t$, $\mu(x, s)$ is the set of all pairs (s, s') such that performing a in s by x leads to s' .

Semantics of BDI

A *Kripke structure* for the language $\mathcal{L}$ of BDI is a system

$$\Sigma = (S, <, B, D, I, m, \mu)$$

such that

- $S \neq \emptyset$ is a set of states
- $< \subseteq S \times S$ is a temporal ordering
- $B, D : Ag \rightarrow 2^{S \times S}$; $B \subseteq D$ (*an agent's desires are his beliefs*)
- $I : Ag \times S \rightarrow 2^{S^*}$ — for $x \in Ag$ and for $s \in S$, $I(x, s)$ is the set of “preferable paths” for x in s ; it is assumed that $I(x, s) \neq \emptyset$ and for every $\wp \in I(x, s)$, $\wp \in S_s^*$
- $m : Var \times S \rightarrow \{0, 1\}$
- $\mu : Ag \times \mathcal{A}t \rightarrow 2^{S \times S}$; for $x \in Ag$ and $a \in \mathcal{A}t$, $\mu(x, s)$ is the set of all pairs (s, s') such that performing a in s by x leads to s' .

Semantics of BDI

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $s \in S$, $\wp \in S^*$.

• $\Sigma, \wp, s \models p$ iff $(s \in \wp) \ \& \ \Sigma, s \models p$ for every $p \in Var$

Semantics of BDI

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $s \in S$, $\wp \in S^*$.

- $\Sigma, \wp, s \models p$ iff $(s \in \wp) \ \& \ \Sigma, s \models p$ for every $p \in Var$
- $\Sigma, \wp, s \models \bigcirc \alpha$ iff $(s \in \wp) \ \& \ \Sigma, succ(\wp, s) \models \alpha$

Semantics of BDI

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $s \in S$, $\wp \in S^*$.

- $\Sigma, \wp, s \models p$ iff $(s \in \wp) \ \& \ \Sigma, s \models p$ for every $p \in Var$
- $\Sigma, \wp, s \models \bigcirc \alpha$ iff $(s \in \wp) \ \& \ \Sigma, succ(\wp, s) \models \alpha$
- $\Sigma, \wp, s \models F\alpha$ iff $(s \in \wp) \ \& \ (\exists s' \in S) \ s' \in \wp \ \& \ s < s' \ \& \ \Sigma, \wp, s' \models \alpha$

Semantics of BDI

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $s \in S$, $\wp \in S^*$.

- $\Sigma, \wp, s \models p$ iff $(s \in \wp) \ \& \ \Sigma, s \models p$ for every $p \in Var$
- $\Sigma, \wp, s \models \bigcirc \alpha$ iff $(s \in \wp) \ \& \ \Sigma, succ(\wp, s) \models \alpha$
- $\Sigma, \wp, s \models F\alpha$ iff $(s \in \wp) \ \& \ (\exists s' \in S) \ s' \in \wp \ \& \ s < s' \ \& \ \Sigma, \wp, s' \models \alpha$
- $\Sigma, \wp, s \models G\alpha$ iff $(s \in \wp) \ \& \ (\forall s' \in S) \ s' \in \wp \ \& \ s < s' \implies \Sigma, \wp, s' \models \alpha$

Semantics of BDI

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $s \in S$, $\wp \in S^*$.

- $\Sigma, \wp, s \models p$ iff $(s \in \wp) \ \& \ \Sigma, s \models p$ for every $p \in Var$
- $\Sigma, \wp, s \models \bigcirc \alpha$ iff $(s \in \wp) \ \& \ \Sigma, succ(\wp, s) \models \alpha$
- $\Sigma, \wp, s \models F\alpha$ iff $(s \in \wp) \ \& \ (\exists s' \in S) \ s' \in \wp \ \& \ s < s' \ \& \ \Sigma, \wp, s' \models \alpha$
- $\Sigma, \wp, s \models G\alpha$ iff $(s \in \wp) \ \& \ (\forall s' \in S) \ s' \in \wp \ \& \ s < s' \implies \Sigma, \wp, s' \models \alpha$
- $\Sigma, \wp, s \models x[a]\alpha$ iff $(s \in \wp) \ \& \ (\forall s' \in S) \ s' \in \wp \ \& \ (s, s') \in \mu(x, a) \implies \Sigma, \wp, s' \models \alpha$

Semantics of BDI

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $s \in S$, $\wp \in S^*$.

- $\Sigma, \wp, s \models p$ iff $(s \in \wp) \ \& \ \Sigma, s \models p$ for every $p \in Var$
- $\Sigma, \wp, s \models \bigcirc \alpha$ iff $(s \in \wp) \ \& \ \Sigma, succ(\wp, s) \models \alpha$
- $\Sigma, \wp, s \models F\alpha$ iff $(s \in \wp) \ \& \ (\exists s' \in S) \ s' \in \wp \ \& \ s < s' \ \& \ \Sigma, \wp, s' \models \alpha$
- $\Sigma, \wp, s \models G\alpha$ iff $(s \in \wp) \ \& \ (\forall s' \in S) \ s' \in \wp \ \& \ s < s' \implies \Sigma, \wp, s' \models \alpha$
- $\Sigma, \wp, s \models x[a]\alpha$ iff $(s \in \wp) \ \& \ (\forall s' \in S) \ s' \in \wp \ \& \ (s, s') \in \mu(x, a) \implies \Sigma, \wp, s' \models \alpha$
- $\Sigma, \wp, s \models x\langle a \rangle \alpha$ iff $(s \in \wp) \ \& \ (\exists s' \in S) \ s' \in \wp \ \& \ (s, s') \in \mu(x, a) \implies \Sigma, \wp, s' \models \alpha$.

Semantics of BDI (cont.)

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $x \in Ag$, $s \in S$.

• $\Sigma, s \models p$ iff $m(p, s) = 1$ for every $p \in Var$

Semantics of BDI (cont.)

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $x \in Ag$, $s \in S$.

- $\Sigma, s \models p$ iff $m(p, s) = 1$ for every $p \in Var$
- $\Sigma, s \models B(x, \alpha)$ iff $(\forall s' \in S) s' \in B(x, s) \implies \Sigma, s' \models \alpha$

Semantics of BDI (cont.)

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $x \in Ag$, $s \in S$.

- $\Sigma, s \models p$ iff $m(p, s) = 1$ for every $p \in Var$
- $\Sigma, s \models B(x, \alpha)$ iff $(\forall s' \in S) s' \in B(x, s) \implies \Sigma, s' \models \alpha$
- $\Sigma, s \models D(x, \alpha)$ iff $(\forall s' \in S) s' \in D(x, s) \implies \Sigma, s' \models \alpha$

Semantics of BDI (cont.)

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $x \in Ag$, $s \in S$.

- $\Sigma, s \models p$ iff $m(p, s) = 1$ for every $p \in Var$
- $\Sigma, s \models B(x, \alpha)$ iff $(\forall s' \in S) s' \in B(x, s) \implies \Sigma, s' \models \alpha$
- $\Sigma, s \models D(x, \alpha)$ iff $(\forall s' \in S) s' \in D(x, s) \implies \Sigma, s' \models \alpha$
- $\Sigma, s \models \alpha$ iff $(\exists \wp \in S_s^*) \Sigma, \wp, s \models \alpha$

Semantics of BDI (cont.)

Let $\Sigma = (S, <, B, D, I, m, \mu)$, $x \in Ag$, $s \in S$.

- $\Sigma, s \models p$ iff $m(p, s) = 1$ for every $p \in Var$
- $\Sigma, s \models B(x, \alpha)$ iff $(\forall s' \in S) s' \in B(x, s) \implies \Sigma, s' \models \alpha$
- $\Sigma, s \models D(x, \alpha)$ iff $(\forall s' \in S) s' \in D(x, s) \implies \Sigma, s' \models \alpha$
- $\Sigma, s \models \alpha$ iff $(\exists \wp \in S_s^*) \Sigma, \wp, s \models \alpha$
- $\Sigma, s \models I(x, \alpha)$ iff $(\forall \wp \in S_s^*) \wp \in I(x, s) \implies \Sigma, \wp, s \models F\alpha$.

Properties of B

Remark: Every symmetric and Euclidean relation is transitive.

- $\models B(x, \alpha) \wedge B(x, \alpha \rightarrow \beta) \rightarrow B(x, \beta)$

Properties of B

Remark: Every symmetric and Euclidean relation is transitive.

- $\models B(x, \alpha) \wedge B(x, \alpha \rightarrow \beta) \rightarrow B(x, \beta)$

- $\models B(x, \alpha) \rightarrow \neg B(x, \neg\alpha)$

(follows from seriality of B)

Properties of B

Remark: Every symmetric and Euclidean relation is transitive.

- $\models B(x, \alpha) \wedge B(x, \alpha \rightarrow \beta) \rightarrow B(x, \beta)$

- $\models B(x, \alpha) \rightarrow \neg B(x, \neg\alpha)$

(follows from seriality of B)

- $\models B(x, \alpha) \rightarrow B(x, B(x, \alpha))$

(follows from transitivity of B)

Properties of B

Remark: Every symmetric and Euclidean relation is transitive.

- $\models B(x, \alpha) \wedge B(x, \alpha \rightarrow \beta) \rightarrow B(x, \beta)$
- $\models B(x, \alpha) \rightarrow \neg B(x, \neg\alpha)$
(follows from seriality of B)
- $\models B(x, \alpha) \rightarrow B(x, B(x, \alpha))$
(follows from transitivity of B)
- $\models \neg B(x, \alpha) \rightarrow B(x, \neg B(x, \alpha))$
(follows from symmetry and transitivity of B).

Properties of D

- $\models D(x, \alpha) \wedge D(x, \alpha \rightarrow \beta) \rightarrow D(x, \beta)$

Properties of D

- $\models D(x, \alpha) \wedge D(x, \alpha \rightarrow \beta) \rightarrow D(x, \beta)$
- $\models D(x, \alpha) \rightarrow \neg D(x, \neg \alpha)$

Properties of D

- $\models D(x, \alpha) \wedge D(x, \alpha \rightarrow \beta) \rightarrow D(x, \beta)$
- $\models D(x, \alpha) \rightarrow \neg D(x, \neg \alpha)$
- $\models D(x, \alpha) \rightarrow B(x, \alpha).$

Properties of I

- **satisfiability:** $\models I(x, \alpha) \rightarrow EF\alpha$ for every $x \in Ag$.

Properties of I

- **satisfiability:** $\models I(x, \alpha) \rightarrow EF\alpha$ for every $x \in Ag$.
- $\models I(x, \alpha) \wedge I(x, \beta) \rightarrow I(x, F\alpha \wedge F\beta)$

If x has an intention α and he has an intention β , then his intention is to achieve both α and β in some (unspecified) order: firstly α then β , or conversely: first β then α .

Properties of I

- **satisfiability:** $\models I(x, \alpha) \rightarrow EF\alpha$ for every $x \in Ag$.

- $\models I(x, \alpha) \wedge I(x, \beta) \rightarrow I(x, F\alpha \wedge F\beta)$

If x has an intention α and he has an intention β , then his intention is to achieve both α and β in some (unspecified) order: firstly α then β , or conversely: first β then α .

- $EG(I(x, \alpha) \wedge \neg\alpha)$ is satisfiable! This formula states that at some state s there exists a path $\wp$ starting in s such that in every state on $\wp$ both $\neg\alpha$ and $I(x, \alpha)$ hold. Therefore, for every state s' on $\wp$ there is a “preferable” path $\wp'$ (starting in s') such that at some state on $\wp'$ α holds. At the same time, in the state s' α does not hold.

Concluding, it reflects the fact: an agent always has some intentions (to achieve goals he aims to), but not necessarily he achieves these goals.

Thank you for your attention!

Any questions are welcome.